

TRABAJO PRÁCTICO

N°1

INTEGRANTES (GRUPO 4):

Martín Caballero
Facundo Galante
Juan Cruz Reveco
Juan Simón Ledda
Bruno Tejada

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

1. Traza del Estado Interno

Para entender el comportamiento anómalo que no lanza excepciones pero falla en producción, debemos analizar paso a paso la gestión de memoria de la estructura HashSet:

- Al ejecutar `tarjetasValidas.add(t1)`:
 - La clase HashSet evalúa internamente al objeto invocando el método sobrescrito `t1.hashCode()`.
 - Este método calcula un valor entero (hash) a partir de los datos que conforman el estado del objeto en ese instante: `codigo = "ACC-101"` y `nivelAcceso = 2`.
 - Utilizando ese valor hash, la colección realiza una operación matemática para determinar el índice del "cajón" o "bucket" en el arreglo interno donde se ubicará el elemento.
 - Como ese espacio en memoria está vacío (al ser el primer elemento), el objeto `t1` se almacena correctamente en dicho bucket.
- Al ejecutar `t1.setNivelAcceso(5)`:
 - Se modifica directamente el estado interno del objeto en memoria a través de una tarjeta de identificación mutable, cambiando el campo `nivelAcceso` de 2 a 5.
 - Este cambio rompe el contrato de las colecciones basadas en hash porque ocurre *después* de que el objeto ya fue insertado.
 - La estructura HashSet es ajena a esta modificación de las propiedades del objeto. El objeto `t1` permanece varado en la ubicación de memoria (el bucket) original, a pesar de que la información por la que fue asignado a ese lugar acaba de cambiar.
- Al ejecutar `tarjetasValidas.contains(t1)`:
 - El método `contains` necesita encontrar el objeto, por lo que vuelve a invocar `t1.hashCode()`.
 - Dado que el método `hashCode` de la clase `TarjetaAcceso` incluye la propiedad `nivelAcceso`, ahora genera un valor hash totalmente diferente correspondiente al nuevo valor (5).
 - El HashSet calcula el bucket usando este *nuevo* hash, que apunta a una ubicación diferente del arreglo interno.
 - Al revisar este nuevo bucket (que probablemente esté vacío o contenga otros objetos distintos), Java concluye equivocadamente que el objeto no está en la colección y devuelve un resultado anómalo (falso) para la consulta.

2. Explicación del Cajón (Bucket)

- ¿En qué número/ubicación de cajón quedó guardado el objeto?
 - El objeto quedó guardado inamoviblemente en el "cajón original".

- Dicho índice inicial fue calculado a partir del valor de hashCode() obtenido con el estado de creación del objeto, es decir, cuando la tarjeta de acceso poseía el nivel 2.
- ¿En cuál cajón lo busca Java al hacer contains()?
 - Java busca en una ubicación incorrecta (el "nuevo cajón").
 - La estructura genera una nueva ruta de búsqueda porque el método hashCode devuelve un número distinto tras la modificación mutada del nivel a 5.
 - En consecuencia, el HashSet revisa un bucket donde el objeto nunca fue introducido y el objeto original se vuelve inalcanzable para las rutinas de búsqueda de la colección, causando el bug silencioso.

3. Fuga de Memoria (Memory Leak): ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?

-Si este patrón se ejecuta repetidamente en un servidor que opera 24/7 genera una fuga de memoria porque cada vez que se modifica un objeto después de insertarlo [cambiar su hashCode()], queda perdido en su cajón original. Al no poder encontrarlo ni borrarlo porque contains() y remove() van al cajón equivocado, entonces el objeto queda atrapado para siempre en el HashSet ocupando memoria y así cada vez se pueden ir acumulando más objetos. En el código podemos observar que esto sucede cuando el nivelAcceso del objeto "t1" vale 2 en un principio y luego se modifica a 5, por lo que cambia su hashCode(). Generando un problema ya que el objeto sigue físicamente almacenado en el bucket correspondiente al hash anterior.

4. Refactorización Obligatoria: Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).

Solución 1: Basada en inmutabilidad

La primera solución consiste en hacer que los atributos utilizados por equals() y hashCode() sean inmutables. De esta manera, una vez que el objeto entra al HashSet, su hash nunca cambia.

```

1  import java.util.Objects;
2
3  class TarjetaAcceso {
4      private final String codigo;
5      private final int nivelAcceso;
6
7      public TarjetaAcceso(String codigo, int nivelAcceso) {
8          this.codigo = codigo;
9          this.nivelAcceso = nivelAcceso;
10     }
11
12     public String getCodigo() {
13         return codigo;
14     }
15
16     public int getNivelAcceso() {
17         return nivelAcceso;
18     }
19
20     @Override
21     public boolean equals(Object o) {
22         if (this == o) return true;
23         if (o == null || getClass() != o.getClass()) return false;
24
25         TarjetaAcceso tarjeta = (TarjetaAcceso) o;
26
27         return nivelAcceso == tarjeta.nivelAcceso &&
28             Objects.equals(codigo, tarjeta.codigo);
29     }
30
31     @Override
32     public int hashCode() {
33         return Objects.hash(codigo, nivelAcceso);
34     }
35 }

```

En este caso se elimina `setNivelAcceso()`. Si se necesita cambiar el nivel de acceso, se debe crear una nueva instancia. La ventaja de esto es que como los valores usados para calcular el `hashCode()` no pueden modificarse, el objeto siempre se encontrará en el bucket correcto.

Solución 2: Ajustando el criterio de la entidad

Otra solución es considerar que la identidad de una tarjeta está determinada únicamente por su código, ya que el nivel de acceso puede cambiar sin que la tarjeta deje de ser la misma y no participa en `hashCode()` ni en `equals()`, el `HashSet` puede seguir encontrándola.

```

1  import java.util.Objects;
2
3  class TarjetaAcceso {
4      private final String codigo;
5      private int nivelAcceso;
6
7      public TarjetaAcceso(String codigo, int nivelAcceso) {
8          this.codigo = codigo;
9          this.nivelAcceso = nivelAcceso;
10     }
11
12     public void setNivelAcceso(int nivelAcceso) {
13         this.nivelAcceso = nivelAcceso;
14     }
15
16     @Override
17     public boolean equals(Object o) {
18         if (this == o) return true;
19         if (o == null || getClass() != o.getClass()) return false;
20
21         TarjetaAcceso tarjeta = (TarjetaAcceso) o;
22
23         return Objects.equals(codigo, tarjeta.codigo);
24     }
25
26     @Override
27     public int hashCode() {
28         return Objects.hash(codigo);
29     }
30 }
31

```

Y así funcionará correctamente porque el atributo que determina el hashCode() osea el código nunca cambia.

Además en las dos soluciones utilizamos *private final* para garantizar que no se puedan modificar luego de crearse.

Consigna 2: Selección y diseño de arquitecturas de colecciones

Requerimiento 1: Sistema de Auditoría de Transacciones

Código:

```
J AuditoriaTransacciones.java
1  import java.util.*;
2
3  class Transaccion {
4
5      private String id;
6      private String descripcion;
7      private long timestamp;
8
9      public Transaccion(String id, String descripcion) {
10
11          this.id = id;
12          this.descripcion = descripcion;
13          this.timestamp = System.currentTimeMillis();
14      }
15
16      @Override
17      public String toString() {
18
19          return "[" + timestamp + "] " + id + " - " + descripcion;
20      }
21  }
22
23  public class AuditoriaTransacciones {
24      public static void main(String[] args) {
25
26          List<Transaccion> registroAuditoria = new ArrayList<>();
27
28          registroAuditoria.add(new Transaccion("T001", "Compra procesada"));
29          registroAuditoria.add(new Transaccion("T002", "Reembolso emitido"));
30          registroAuditoria.add(new Transaccion("T003", "Pago confirmado"));
31
32          for (Transaccion t : registroAuditoria) {
33              System.out.println(t);
34          }
35      }
36  }
```

Justificación: Elegimos usar un ArrayList (con la interfaz List) porque un registro de auditoría funciona como un historial: la idea es simplemente ir anotando las transacciones a medida que ocurren. Esta estructura es perfecta porque nos permite agregar elementos al final de la lista de una manera muy rápida. Además, respeta el orden exacto en el que fuimos guardando todo, lo cual es ideal para después poder leer e imprimir el historial de principio a fin sin complicaciones.

Requerimiento 2: Catálogo de ofertas relámpago

Código:

```
J CatalogoOfertas.java
1  import java.time.LocalDateTime;
2  import java.util.*;
3
4  class Producto {
5
6      private String id;
7      private String nombre;
8      private long precio;
9      private LocalDateTime fechaVencimiento;
10
11     public Producto(String id, String nombre, long precio,
12                     LocalDateTime fechaVencimiento) {
13
14         this.id = id;
15         this.nombre = nombre;
16         this.precio = precio;
17         this.fechaVencimiento = fechaVencimiento;
18     }
19
20     public String getId() { return id; }
21     public String getNombre() { return nombre; }
22     public long getPrecio() { return precio; }
23
24     public boolean estaVigente() {
25         return LocalDateTime.now().isBefore(fechaVencimiento);
26     }
27
28     @Override
29     public String toString() {
30
31         return nombre + " ($" + precio + ", vence: " + fechaVencimiento + ")";
32     }
33 }
34
35 public class CatalogoOfertas {
36
37     public static void main(String[] args) {
38
39         NavigableSet<Producto> catalogo = new TreeSet<>(
40             Comparator.comparingLong(Producto::getPrecio)
41                 .thenComparing(Producto::getId)
42         );
43
44         catalogo.add(new Producto("P1", "Auriculares", 15000,
45                                 LocalDateTime.now().plusHours(2)));
46         catalogo.add(new Producto("P2", "Mouse", 15000,
47                                 LocalDateTime.now().plusHours(1)));
48         catalogo.add(new Producto("P3", "Notebook", 850000,
49                                 LocalDateTime.now().minusMinutes(1)));
50     }
```

```

51      System.out.println("Más barato: " + catalogo.first());
52      System.out.println("Más caro: " + catalogo.last());
53      System.out.println("Catálogo ordenado: " + catalogo);
54
55      System.out.println("Ofertas vigentes:");
56      for (Producto producto : catalogo) {
57          if (producto.estaVigente()) {
58              System.out.println(producto);
59          }
60      }
61  }
62  }

```

Justificación: Para el catálogo de ofertas usamos un TreeSet (bajo la interfaz NavigableSet) porque necesitábamos que los productos se ordenaran automáticamente por precio y asegurarnos de que no hubiera repetidos. Esta estructura es excelente porque mantiene todo organizado a medida que agregamos cosas y lo hace de forma rápida. La mayor ventaja es que, al usar la interfaz NavigableSet, nos da herramientas súper útiles para encontrar directamente el producto más barato y el más caro, sin tener que revisar la lista completa uno por uno. Para hacerlo había usado la interfaz Set pero Copilot me recomendó usar NavigableSet. Y consultando con otras IAs y comparando las dos interfaces me explicaron que NavigableSet es una evolución de Set, que te permite hacer consultas de aproximación. Podes pedirle no sólo un valor exacto, sino los valores "más cercanos" a un punto de referencia y además poder acceder y eliminar con más eficiencia a los elementos de cada extremo, el elemento más pequeño o el más grande, como requería la consigna.

Requerimiento 3: Búsqueda de sesiones de usuarios activos

Código:


```

1  import java.util.HashMap;
2  import java.util.Map;
3
4  public class SessionManager {
5
6      private Map<String, String> activeSessions = new HashMap<>();
7
8      public void login(String token, String username) {
9          activeSessions.put(token, username);
10     }
11
12     public String buscarSesion(String token) {
13         return activeSessions.get(token);
14     }
15
16     public void logout(String token) {
17         activeSessions.remove(token);
18     }
19
20     public static void main(String[] args) {
21         SessionManager manager = new SessionManager();
22
23         manager.login("TOKEN-A1B2", "jperez");
24         manager.login("TOKEN-C3D4", "mgomez");
25
26         String usuario = manager.buscarSesion("TOKEN-C3D4");
27         System.out.println("Usuario encontrado: " + usuario);
28
29         manager.logout("TOKEN-A1B2");
30     }
31 }
32

```

Justificación:

Utilizamos “HashMap” porque de acuerdo al requerimiento se debe buscar y encontrar instantáneamente la sesión de un usuario dado un token. Y sabemos que “HashMap” guarda pares clave-valor (token-usuario) y accede a ellos con complejidad $O(1)$ promedio, usando el hash del token para ubicarlo directamente, sin recorrer nada.

Elegimos “Map” en vez de “List”, “Set” o “Queue” que procesa elementos de forma FIFO (el primero que entra es el primero que sale), porque necesitamos asociar una clave (token) con un valor (usuario), no solo guardar elementos sueltos.

Con 500.000 usuarios conectados simultáneamente la búsqueda sigue siendo prácticamente instantánea sin importar cuántas sesiones haya activas.

Requerimientos	Interfaz seleccionada	Implementación concreta	Complejidad de búsqueda	Complejidad de inserción
Requerimiento 1	List	ArrayList	$O(n)$	$O(1)$ amortizada
Requerimiento 2	Set (NavigableSet)	TreeSet	$O(\log n)$	$O(\log n)$
Requerimiento 3	Map	HashMap	$O(1)$ promedio	$O(1)$ promedio

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

1.

```

8
9 public class consigna_3 {
10     public static Character primerCaracterNoRepetido(String texto) {
11
12         Map<Character, Boolean> repetido = new HashMap<>();
13
14         for (int i = 0; i < texto.length(); i++) {
15             char c = texto.charAt(i);
16             if (repetido.containsKey(c)) {
17                 repetido.put(c, value: true);
18             } else {
19                 repetido.put(c, value: false);
20             }
21         }
22
23         for (int i = 0; i < texto.length(); i++) {
24             char c = texto.charAt(i);
25             if (!repetido.get(c)) {
26                 return c;
27             }
28         }
29
30         return null;
31     }
32 }
33
34 Run | Debug
35 public static void main(String[] args) {
36     String texto = "barco";
37     String texto2 = "abarco";
38     System.out.println(primerCaracterNoRepetido(texto));
39     System.out.println(primerCaracterNoRepetido(texto2));
40
41
42
43 }

```

Problems Output Debug Console Terminal Ports Filter (e.g. text, lexcludeTex... Code

1 error

[Done] exited with code=1 in 0.583 seconds

[Running] cd "/Users/martin.caballero/Documents/UTN/Desarrollo de software/HTML/" && javac consigna_3.java && java consigna_3

b

b

[Done] exited with code=0 in 0.626 seconds

2.

Si dos caracteres diferentes generan el mismo hashCode(), se produce una colisión. Java resuelve esto mediante una técnica llamada *separate chaining* (encadenamiento separado), almacenando ambas claves en el mismo bucket y diferenciándose mediante la comparación de las claves, además del hash. De esta forma, aunque compartan el mismo hash, cada carácter mantiene su propio valor y no se pierde información. En caso de muchas colisiones, Java 8+ puede convertir el bucket en un árbol balanceado para mejorar el rendimiento

Consigna 4: Caso Teórico de Borde y Colisiones

Se toman las cadenas s1 = "FB" y s2 = "Ea", con s1.hashCode() == s2.hashCode() == 2236

1.

La colección final contendrá 2 elementos. Java NUNCA determina la igualdad de dos objetos usando solamente hashCode(); el hashCode únicamente sirve para decidir en qué cajón buscar o insertar. El proceso interno es el siguiente:

- Se inserta s1 = "FB". Se calcula su hashCode() (2236), se aplica la función de dispersión y, con la capacidad por defecto de 16 cajones, se ubica en el cajón número 12. Se guarda ahí.
- Se intenta insertar s2 = "Ea". Como s2.hashCode() también es 2236, el spread da el mismo resultado y el algoritmo también apunta al cajón 12 (el mismo que ya ocupa s1): esto es una colisión de hash.
- Antes de agregar s2, Java recorre las entradas que ya existen en el cajón 12 y le aplica equals() a cada una. Se evalúa s1.equals(s2) → "FB".equals("Ea") → false, porque el contenido de caracteres es distinto (aunque el hash sea igual).
- Como ninguna entrada existente es equals() a s2, Java concluye que s2 es un elemento nuevo y lo agrega igualmente al cajón 12, a continuación de s1 (encadenado).

Conclusión: el HashSet queda con size() == 2. El hashCode() compartido solo obliga a ambos elementos a convivir en el mismo cajón; es equals() quien decide, con la comparación de contenido, si se trata del mismo elemento o de dos elementos distintos que simplemente colisionaron.

2.

En el caso ideal, cada cajón de un HashMap contiene 0 o 1 entradas, por lo que `get()/put()` son $O(1)$ en promedio: el hash lleva directamente al cajón correcto. Si muchas claves colisionan y se acumulan en el mismo cajón en el peor escenario posible, si TODAS las claves cayeran en un único cajón, ese cajón pasa a comportarse como una estructura secuencial de n elementos:

- Sin treeificación (listas enlazadas, comportamiento clásico pre-Java 8): buscar una clave dentro de ese cajón exige recorrer la lista comparando con `equals()` elemento por elemento, degradando la complejidad de $O(1)$ a $O(n)$ en el peor caso.
- Con treeificación (Java 8 en adelante): si un cajón acumula 8 o más entradas y la tabla tiene capacidad ≥ 64 , esa lista se convierte en un árbol rojo-negro balanceado, lo que mejora el peor caso de ese cajón de $O(n)$ a $O(\log n)$.

En cualquiera de los dos casos, acumular colisiones siempre empeora el rendimiento respecto del $O(1)$ promedio esperado; la treeificación de Java 8+ solo amortigua el impacto (de lineal a logarítmico) sin eliminarlo, motivo por el cual una buena función de `hashCode()` que distribuya uniformemente las claves entre los cajones sigue siendo crítica para el rendimiento real de un HashMap.